

Lenguajes de Programacion

Alto Nivel y Bajo Nivel

Estudiante:

Angel Fernando Calero Maldonado

Universidad Estatal de Milagro

UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO

Que es un Lenguaje de Programacion?

Un **lenguaje de programacion** es el medio formal que permite a un ser humano comunicar instrucciones a una computadora. Segun su nivel de abstraccion respecto al hardware, se clasifican en dos grandes categorias:

Alto Nivel

Proximo al lenguaje humano. Abstracto, legible y portable. El programador no necesita conocer la arquitectura interna del procesador.

Bajo Nivel

Proximo al lenguaje maquina. Controla directamente el hardware. Requiere conocimiento especializado en arquitectura computacional.

“A mayor abstraccion, mayor cercania al razonamiento humano y mayor distancia del hardware.”

Un **lenguaje de alto nivel** esta disenado para ser comprensible por los seres humanos. Su sintaxis se asemeja al lenguaje natural, permitiendo al programador enfocarse en la logica del problema sin preocuparse por los detalles del hardware.

Caracteristicas principales

- ▶ **Abstraccion del hardware:** oculta registros, direcciones de memoria y llamadas al sistema operativo.
- ▶ **Portabilidad:** el mismo codigo se ejecuta en distintas plataformas con minimas modificaciones.
- ▶ **Gestion automatica de memoria:** muchos incluyen *garbage collection*.
- ▶ **Bibliotecas y *frameworks*:** ecosistema amplio que acelera el desarrollo.

Ejemplos: Python, Java, JavaScript, C#, Ruby, PHP.

Un **lenguaje de bajo nivel** opera cerca del lenguaje maquina. Sus instrucciones se corresponden de forma directa con las operaciones ejecutadas por el procesador. La abstraccion es minima o nula.

Lenguaje Maquina

Primer nivel: instrucciones representadas en binario (0s y 1s). Es el unico lenguaje que el procesador ejecuta de forma nativa, sin ninguna traduccion intermedia.

Ejemplos: NASM (x86/Linux), MASM (Windows), GAS, ARM Assembly.

Lenguaje Ensamblador

Segundo nivel: usa codigos mnemotecnicos como MOV, ADD, INT. Requiere un *ensamblador* para traducirse a codigo maquina binario.

✓ Ventajas

- ▶ Sintaxis legible y cercana al lenguaje humano.
- ▶ Desarrollo mas rapido y menos propenso a errores.
- ▶ Gran portabilidad entre plataformas.
- ▶ Comunidad amplia y abundante documentacion.
- ▶ Gestion automatica de recursos del sistema.

✗ Desventajas

- ▶ Menor eficiencia respecto al codigo maquina.
- ▶ Mayor consumo de memoria por la abstraccion.
- ▶ Menor control sobre el hardware.
- ▶ Dependencia del compilador o interprete.
- ▶ No apto para programar drivers o kernels.

✓ Ventajas

- ▶ Control total sobre el hardware y los recursos.
- ▶ Maxima eficiencia y velocidad de ejecucion.
- ▶Codigo altamente optimizado para tareas criticas.
- ▶ Indispensable para sistemas embebidos y kernels.
- ▶ Sin dependencia de traductores intermedios.

✗ Desventajas

- ▶ Curva de aprendizaje muy elevada.
- ▶Codigo dificil de leer, depurar y mantener.
- ▶ Especifico para cada arquitectura (x86, ARM, MIPS).
- ▶ Mayor riesgo de errores criticos de memoria.
- ▶ Desarrollo lento y costoso en tiempo.

El siguiente programa imprime un mensaje en pantalla utilizando **Python**, lenguaje de alto nivel reconocido por su sintaxis clara y concisa:

```
^^I^^I^^I# Programa en Python - Alto Nivel
^^I^^I^^I# Imprime un mensaje en pantalla
^^I^^I^^I
^^I^^I^^Imensaje = "Hola, Mundo!"
^^I^^I^^Iprint(mensaje)
^^I^^I^^I
```

Observacion

Con **una sola instruccion** (`print`) el programador ordena mostrar texto en pantalla. El lenguaje gestiona internamente la llamada al sistema operativo, los *buffers* de salida y la codificación del texto. El programador no necesita conocer ninguno de esos detalles.

El mismo procedimiento expresado en **pseudocodigo**: representacion abstracta que describe la logica con lenguaje natural estructurado, independiente de cualquier lenguaje o plataforma:

```
^^I^^I^^IINICIO
^^I^^I^^IDEFINIR mensaje COMO CADENA
^^I^^I^^Imensaje <- "Hola, Mundo!"
^^I^^I^^IESCRIBIR mensaje
^^I^^I^^IFIN
^^I^^I
```

Proposito del Pseudocodigo

Permite disenar la logica de un algoritmo antes de codificarlo en un lenguaje real. No requiere compilacion ni interprete. Es la herramienta pedagogica fundamental en la ensenanza de algoritmos.

La misma tarea en **lenguaje ensamblador** requiere configurar manualmente los registros del procesador e invocar una interrupcion del sistema operativo:

```
^^I^^I^^Isection .data
^^I^^I^^Imsg db "Hola, Mundo!", 0x0A
^^I^^I^^Ilen equ $ - msg
^^I^^I^^I
^^I^^I^^Isection .text
^^I^^I^^Iglobal _start
^^I^^I^^I_start:
^^I^^I^^Imov eax, 4      ; syscall sys_write
^^I^^I^^Imov ebx, 1      ; stdout
^^I^^I^^Imov ecx, msg    ; direccion del mensaje
^^I^^I^^Imov edx, len    ; longitud del mensaje
^^I^^I^^Iint 0x80        ; interrupcion al kernel
^^I^^I^^I
^^I^^I^^Imov eax, 1      ; syscall sys_exit
^^I^^I^^Ixor ebx, ebx    ; codigo de salida 0
^^I^^I^^Iint 0x80        ; interrupcion al kernel
^^I^^I^^I
```

Por debajo del ensamblador existe el **lenguaje maquina**: secuencia de bits que el procesador ejecuta directamente. Cada instruccion se traduce a valores hexadecimales y binarios:

```
^^I^^I^^IInstruccion: int 0x80
^^I^^I^^IHex      : CD 80
^^I^^I^^IBinario: 11001101 10000000
^^I^^I^^I
^^I^^I^^IInstruccion: mov eax, 4
^^I^^I^^IHex      : B8 04 00 00 00
^^I^^I^^IBinario: 10111000 00000100 00000000 00000000 00000000
^^I^^I^^I
^^I^^I^^IInstruccion: mov ebx, 1
^^I^^I^^IHex      : BB 01 00 00 00
^^I^^I^^IBinario: 10111011 00000001 00000000 00000000 00000000
^^I^^I^^I
```

Reflexion

Es el unico lenguaje que la CPU comprende de forma nativa. El ensamblador surgio para liberar al programador de escribir directamente en binario.

Tarea: Imprimir “Hola, Mundo!” en pantalla

Alto Nivel (Python)

```
print("Hola, Mundo!")
```

1 linea de codigo

Ensamblador (NASM)

Definir seccion de datos
Cargar registros eax, ebx
Asignar ecx y edx
Invocar interrupcion 0x80

≈14 lineas de codigo

Lenguaje Maquina

B8 04 00 00 00
BB 01 00 00 00
CD 80 ...

Solo 0s y 1s

La misma tarea requiere un nivel de detalle radicalmente distinto segun la abstraccion del lenguaje utilizado.

Lenguajes de Alto Nivel – Python, Java, JavaScript, C#

Lenguajes de Nivel Medio – C, C++

Lenguaje Ensamblador – NASM, MASM, GAS

Lenguaje Maquina –Codigo Binario (0s y 1s)

Hardware – Procesador (CPU)

Mayor
abstraccion



Se usa Alto Nivel cuando...

- ▶ Se desarrollan aplicaciones web, móviles o de escritorio.
- ▶ El tiempo de desarrollo es crítico.
- ▶ Se trabaja con inteligencia artificial o ciencia de datos.
- ▶ La portabilidad entre plataformas es necesaria.
- ▶ El equipo de trabajo es amplio y diverso.

Se usa Bajo Nivel cuando...

- ▶ Se programan sistemas operativos o kernels.
- ▶ Se desarrollan controladores de hardware (drivers).
- ▶ Se trabaja con sistemas embebidos o IoT.
- ▶ La eficiencia máxima de ejecución es imprescindible.
- ▶ Se realiza ingeniería inversa o análisis de malware.

Sintesis

- ▶ Los lenguajes de **alto nivel** priorizan la productividad del desarrollador y la legibilidad, a costa de menor control del hardware.
- ▶ Los lenguajes de **bajo nivel** priorizan el rendimiento y el control directo del procesador, a costa de mayor complejidad.
- ▶ **Ninguno reemplaza al otro**: coexisten y se complementan segun el contexto del proyecto.
- ▶ La comprension del bajo nivel permite escribir codigo de alto nivel mas eficiente y consciente de los recursos del sistema.

“Conocer como funciona la maquina a nivel profundo convierte a un programador de alto nivel en un mejor ingeniero de software.”

- ▶ Chakray (2025). *Lenguajes de programacion: tipos, características y diferencias*. Recuperado de: chakray.com
- ▶ Niauxus (2024). *Lenguajes de programacion de alto nivel: que son*. Recuperado de: niauxus.com
- ▶ Lenovo Mexico (2024). *Lenguaje de alto nivel: todo lo que necesitas saber*. Recuperado de: lenovo.com/mx
- ▶ MSMK University (2025). *El lenguaje Assembler*. Recuperado de: msmk.university
- ▶ Acevedo Maldonado, J. (2023). *Hola mundo en ensamblador x86*. Medium / Nabucodonosor Editorial.
- ▶ UNAM – UAPA (s.f.). *Lenguajes de Programacion: clasificacion y características*. Repositorio CUAED.
- ▶ LinkedIn Advice (2024). *High-level vs low-level programming languages*. Recuperado de: linkedin.com

www.unemi.edu.ec

Gracias

UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO